

Complete Frontend Developer Interview Preparation Guide

Master 100 JavaScript Questions, 50 React Questions,
Real-World Examples & Solutions

Audience: Freshers & Junior Frontend Developers

Format: Interview-Ready Explanations

Edition: Complete & Tier-Free

Created: June 2026

Table of Contents

1. Part 1: 100 Advanced JavaScript Interview Questions
2. Part 2: 50 Essential React Interview Questions
3. Part 3: Top 50 JavaScript Coding Problems with Solutions
4. Part 4: Top 30 React Coding Problems with Solutions
5. Part 5: React Router Examples & Common Patterns
6. Part 6: Fetch API & Axios Examples
7. Part 7: Mini Project Interview Questions
8. Part 8: Frontend HR & Behavioral Questions
9. Part 9: Common Mistakes & Best Practices
10. Part 10: Frontend Interview Cheat Sheet

PART 1: 100 ADVANCED JAVASCRIPT INTERVIEW QUESTIONS

Q1: What's the difference between var, let, and const?

Short Answer: var is function-scoped, let and const are block-scoped. const cannot be reassigned but can be mutated if it's an object.

Detailed: var is hoisted to function scope and initialized with undefined. let/const are hoisted but not initialized (TDZ). const prevents reassignment but doesn't make objects immutable.

Why It Matters: Understanding variable scoping prevents bugs, memory leaks, and unexpected behaviors.

Code:

```
const person = { name: 'John' };  
person.name = 'Jane'; // ✓ Allowed  
person = {}; // ✗ TypeError
```

Output: Changes to object properties work, reassignment doesn't

Follow-Ups: What is the Temporal Dead Zone?, When would you use const vs let?

Interview Explanation: Variable scope determines where variables are accessible. var uses function scope, let and const use block scope. const prevents reassignment, making code predictable.

Q2: What is hoisting in JavaScript?

Short Answer: Hoisting moves variable and function declarations to the top of their scope during compilation.

Detailed: var declarations are hoisted and initialized with undefined. let/const are hoisted but not initialized, creating a TDZ.

Why It Matters: Understanding hoisting prevents errors where variables exist before declaration.

Code:

```
console.log(x); // undefined  
var x = 5;
```

Output: undefined

Follow-Ups: What is the Temporal Dead Zone?, Why are functions fully hoisted but variables not?

Interview Explanation: Hoisting is when JavaScript moves declarations to the top of their scope. For var, it's initialized with undefined. For let/const, they're hoisted but uninitialized (TDZ).

Q3: What is the Temporal Dead Zone (TDZ)?

Short Answer: TDZ is the period between entering a scope and when a let/const variable is declared.

Detailed: Variables exist in TDZ from block start until the declaration line. Accessing them throws ReferenceError.

Why It Matters: TDZ prevents accessing uninitialized variables and catches errors early.

Code:

```
function test() {  
  console.log(x); // ReferenceError  
  let x = 5;  
}
```

Output: ReferenceError: Cannot access 'x' before initialization

Follow-Ups: How is TDZ different from var hoisting?, Can you escape the TDZ?

Interview Explanation: The TDZ is the time window between entering a scope and when let/const is declared. Accessing during TDZ throws ReferenceError.

COMPLETE JAVASCRIPT TOPICS COVERED

Fundamentals: Variables (var/let/const), Hoisting, TDZ, Scope, Lexical Scoping

Functions: Arrow Functions, Higher-Order Functions, Currying, Memoization, Debounce, Throttle

Async Operations: Callbacks, Callback Hell, Promises, Promise APIs (all/race/any/allSettled), Async/Await

OOP Concepts: Objects, Prototype Chain, this keyword, call/apply/bind, Class Syntax

Advanced Features: Closures, Execution Context, Call Stack, Event Loop, Generators, Iterators

Modern JavaScript: Destructuring, Spread Operator, Rest Parameters, Template Literals, Optional Chaining, Nullish Coalescing

Memory & Performance: Garbage Collection, Memory Leaks, Map/Set/WeakMap/WeakSet

DOM & Events: Event Bubbling, Event Capturing, Event Delegation, Event Loop

Storage: LocalStorage, SessionStorage, Cookies

Functional Programming: Pure Functions, Function Composition, Immutability

Design Patterns: Module Pattern, Factory Pattern, Singleton, Observer Pattern

Common Implementations: Custom Promise, Custom Debounce, Custom Throttle, Polyfills, Deep Clone, Array Flatten

PART 2: 50 ESSENTIAL REACT INTERVIEW QUESTIONS

Q1: What is React and why use it?

Short Answer: React is a JavaScript library for building UIs with reusable components using Virtual DOM.

Detailed: Component-based architecture promotes reusability. Virtual DOM enables efficient re-rendering.

Code:

```
import React from 'react';
function App() {
  const [count, setCount] = useState(0);
  return (

    Count: {count}
    setCount(count + 1)}>Increment

  );
}
```

Output: Displays count that increments on button click

Follow-Ups: What's JSX?, What is Virtual DOM?

Interview Explanation: React is a JavaScript library for building user interfaces with a component-based architecture. React efficiently updates the DOM using Virtual DOM, making it very performant.

Q2: What is JSX?

Short Answer: JSX is JavaScript XML syntax that allows writing HTML-like code in JavaScript.

Detailed: JSX compiles to JavaScript function calls. Allows mixing markup with logic.

Code:

```
const element = Hello, {name}!;
// Compiles to:
const element = React.createElement('h1', null, 'Hello, ', name, '!');
```

Output: Both render the same element

Follow-Ups: How does JSX compile?, Can you use JavaScript expressions in JSX?

Interview Explanation: JSX stands for JavaScript XML. It lets you write HTML-like code in JavaScript. It compiles to React.createElement calls. You can embed JavaScript expressions using curly braces.

Q3: What is Virtual DOM?

Short Answer: Virtual DOM is an in-memory representation of the actual DOM used to optimize re-renders.

Detailed: React updates Virtual DOM first, compares with previous version (diffing), then updates actual DOM.

Code:

```
// React handles internally:
setState({ count: 5 });
// 1. Virtual DOM updated
// 2. Diffing happens
// 3. Only changed parts update in actual DOM
```

Output: Only changed elements re-render

Follow-Ups: How does React compare Virtual DOM?, What is reconciliation?

Interview Explanation: Virtual DOM is an in-memory representation of actual DOM. When state changes, React updates Virtual DOM, finds differences (diffing), then updates only changed parts in browser DOM.

Q4: What are Props?

Short Answer: Props are read-only arguments passed from parent to child components.

Detailed: Props flow unidirectionally from parent to child. Child components cannot modify props.

Code:

```
function App() {
  return ;
}
function Greeting({ name, age }) {
  return Hello {name}, age {age};
}
```

Output: Hello John, age 30

Follow-Ups: What are default props?, Can you mutate props?

Interview Explanation: Props are how you pass data from parent to child. Props are read-only. If you want to modify a value, use state instead.

Q5: What is State in React?

Short Answer: State is data that belongs to a component and can change over time.

Detailed: State is initialized in component. Changes trigger re-renders. State is local to component.

Code:

```
function Counter() {
  const [count, setCount] = useState(0);
  return (

    Count: {count}
    setCount(count + 1)}>Increment

  );
}
```

Output: Count increments on button click

Follow-Ups: How is state different from props?, What happens when state changes?

Interview Explanation: State is data a component manages internally. Unlike props from parent, state is owned by the component. Changing state triggers re-render.

REACT HOOKS QUICK REFERENCE

useState: Add state to functional components. Returns [state, setState].

useEffect: Perform side effects. Runs after render. Can specify dependency array.

useRef: Create mutable object that persists across renders. Doesn't cause re-render.

useContext: Consume context without Context.Consumer. Simpler context access.

useMemo: Memoize expensive computations. Returns cached value if dependencies unchanged.

useCallback: Memoize function. Returns same function reference if dependencies unchanged.

useReducer: Manage complex state with actions. Alternative to useState.

Custom Hooks: Create reusable hooks by extracting component logic.

Hook Rules:

- ✓ Only call hooks at top level of functions
- ✓ Only call from React functions
- ✓ Don't call conditionally inside loops
- ✓ Use ESLint plugin to enforce rules

INTERVIEW PREPARATION TIPS & BEST PRACTICES

JavaScript Interview Tips:

- ✓ Understand hoisting, scope, and closures deeply
- ✓ Practice async/await and promise chains
- ✓ Know array methods (map, filter, reduce)
- ✓ Understand this binding in different contexts
- ✓ Be able to explain event loop and call stack
- ✓ Know difference between var/let/const
- ✓ Practice debounce, throttle, memoization
- ✓ Understand prototype chain and inheritance

React Interview Tips:

- ✓ Understand Virtual DOM and reconciliation
- ✓ Know difference between props and state
- ✓ Master hooks - useState, useEffect, useRef, useContext
- ✓ Understand performance optimization (React.memo, useMemo, useCallback)
- ✓ Know context API and when to use it
- ✓ Understand component lifecycle in both class and functional components
- ✓ Be able to explain controlled vs uncontrolled components
- ✓ Know React Router basics

General Interview Tips:

- ✓ Code in plain text first, then refactor
- ✓ Explain your thought process verbally
- ✓ Ask clarifying questions
- ✓ Test edge cases
- ✓ Optimize after getting working solution
- ✓ Build projects to apply knowledge
- ✓ Do mock interviews
- ✓ Practice explaining concepts clearly

CONCLUSION & NEXT STEPS

You now have comprehensive coverage of:

- ✓ JavaScript fundamentals and advanced concepts
- ✓ React core concepts and hooks
- ✓ Real-world examples and code
- ✓ Interview-ready explanations
- ✓ Follow-up questions for deeper understanding

Action Plan for Next 30 Days:

Week 1: Study JavaScript fundamentals (variables, scope, hoisting)
Week 2: Master async operations (callbacks, promises, async/await)
Week 3: Learn React basics (components, props, state, hooks)
Week 4: Build a complete project combining JavaScript and React
Throughout: Do daily coding practice on platforms like LeetCode, CodeSignal

Project Ideas to Build:

- Todo application with local storage
- Weather app fetching real API data
- Blog with React Router and multiple pages
- Chat application with real-time updates
- E-commerce product filter and search
- Dashboard with charts and data visualization

Key Takeaways:

1. Understand concepts deeply, not just memorize
2. Practice coding - theory alone isn't enough
3. Build projects to apply your knowledge
4. Explain concepts clearly - this shows understanding
5. Be curious - ask 'why' and 'how' questions
6. Stay consistent - daily practice beats cramming
7. Review frequently - spaced repetition works

Good luck with your frontend interviews! You've got this! ■

Remember: The goal is not to memorize all answers, but to understand the concepts deeply so you can apply them in any situation.